

Advanced Front-end Web Development - Group 5

Reflection about Flexbox & Grid Activity

Level 1:

This activity helped me understand Flexbox and CSS Grid through actually working with them instead of just learning the concepts. I learned how to use Flexbox to arrange my page elements and how Grid can structure the page into rows. At first, I found some of the CSS properties and file organization a little confusing, but working through the steps helped me understand them better.

I also practiced organizing my files, creating a branch, committing my changes, pushing my work, and preparing a Pull Request. Overall, I feel that this activity improved my CSS skills and made me more comfortable with the development workflow and I enjoyed seeing my work come together step by step. I made a few mistakes along the way, but fixing them helped me understand the process better.

Level 2:

Not the most exciting layout in the world, just a single row of two columns with some bars around it.

With flexbox I made the body a column flex container and then the content div had to be its own flex container so the main and sidebar could sit side by side. That nesting bugged me a little, two flex contexts for what is really just one row. The 70/30 split I did with flex-basis, main at 70 and sidebar at 30. It works, and on mobile I just set both to 100% and they stack.

Grid was much cleaner here. The body is one grid with grid-template-areas so header, nav, content and footer are all put out in a single block. Then content is another grid with 7fr 3fr, and I actually like that because you can read the 70/30 straight off the numbers. On mobile the content grid drops to one column and everything stacks by itself.

At this level both were fine, but I would say the easiest was grid because the whole page skeleton is just one grid-template-areas block, while flexbox needed that extra nesting step. Code length came out pretty much the same either way.

What I learnt now is to use grid for the overall page and any real two dimensional layout, flexbox for the small one dimensional rows like the nav.

Level 3:

For level 3, I created the same page using Flexbox and Grid. Aligning the three columns was easy, but changing their positions for tablet and mobile device required more consideration.

In flexbox, I used nested containers and set their sizes, wrapping and order properties. Grid seemed clearer to understand since I could see where the sections go from the named areas. My error when placing inner grid specifications within the outer container led to items being aligned in strange places. Correcting the error taught me how Grid works with its direct children. Padding and margin were revealed to change the position of percentages.

The two layouts required roughly equal amount of code, but Grid was more understandable for me in this case. I would use it for structuring the whole page and Flexbox for simple rows like menus.

Level 4:

The layout for level 4 was a holy grail, with a header, nav, two sidebars around a main content area, and a footer.

The thing that was quite hard to do was the sidebar collapsing on the small screen. In flexbox it was easy, I just set flex-basis to 0 on the sidebar when it gets the is-collapsed class and it shrinks away. Grid was different because the column width lives on the parent container, not on the sidebar itself. So toggling a class on the sidebar did nothing. I had to go back to the JavaScript and also flip a class on the layout wrapper so I could change grid-template-columns. That one small detail made flexbox feel easier for this part.

For the cards and the footer grid was clearly easier to implement and position. The cards are just `repeat(auto-fill, minmax(280px, 1fr))` which is one line and handles the reflow on its own. With flexbox I used flex-wrap and a basis of 280px, which works, but the last card on a row stretches across the whole width and it looks a bit off. The footer was the same, grid's `repeat(3, 1fr)` is cleaner than giving each column flex: 1 1 160px.

Deciding which one was intuitive depends on which section of the UI you look at. The one dimensional stuff like the nav, the header, the buttons inside each card, I kept those as flex even in the grid version, because trying to grid a simple row feels like overkill. But for anything that is a real grid, columns and rows, grid just makes more sense, the named areas especially.

If I had to pick, I'd say grid for the overall page skeleton and any card or link grid, and flexbox for the small horizontal sections. They're not really competing, I think the trick is just knowing which one fits the part you are working on. Code-wise, grid ended up a bit shorter overall, mostly because of the cards and footer.